STUDENT NAME: AMANDEEP	MODULE: My Practice	CODE: UI_009_ASYNC
BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02	COURSE: UI Development with Angular using AI	STATUS: In Progress

Practice Worksheet – 009 – Angular Clock using RxJS & Async Pipe

Stream real-time clock observables using RxJS timer() and interval() operators, rendered in templates via the Angular | async pipe.

KEY LEARNING OBJECTIVES:

- Consume asynchronous RxJS Observable streams using the async pipe.
- Maintain clean separation between component logic, templates, and styling.
- Ensure responsive, accessible, and error-resilient user interfaces.
- Verify correctness with client-side console logs and DOM unit test assertions.

STEP-BY-STEP WORKSHEET INSTRUCTIONS:

Step 1: Review front-end UI requirements and identify user interaction states.
Step 2: Implement HTML structure, TypeScript component classes, and CSS styling.
Step 3: Integrate event listeners, data bindings, or reactive validation rules.
Step 4: Verify rendering in modern browsers and test boundary inputs.

EXERCISE 9.1: COMPONENT ARCHITECTURE & DOM STRUCTURE (RXJS TIMER OBSERVABLE)

Problem Statement: Define the template markup and structural elements for 009 – Angular Clock using RxJS & Async Pipe. Establish proper semantic tags, component selectors, and layout containers.

Reference Baseline / Scenario / Code:

```
<div class="card-container"> <!-- Template structure for 009 – Angular Clock using RxJS & Async Pipe --> </div>
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] HTML Template Markup: _____

[2] Component Selector & Metadata: _____

[3] Semantic Containers Setup: _____

EXERCISE 9.2: TYPESCRIPT / JAVASCRIPT CORE IMPLEMENTATION (ASYNC PIPE TEMPLATE BINDING)

Problem Statement: Implement the core logic for 009 – Angular Clock using RxJS & Async Pipe. Write the TypeScript methods, reactive bindings, or event listener functions handling state changes.

Reference Baseline / Scenario / Code:

```
export class ExampleComponent { // Component logic for 009 – Angular Clock using RxJS & Async Pipe
}
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] TypeScript Class / JS Function: _____

[2] State Variables & Models: _____

[3] Event Handlers & Directives: _____

EXERCISE 9.3: STYLING, VALIDATION & EDGE HANDLING (AUTOMATIC UNSUBSCRIPTION)

Problem Statement: Implement validation logic and styling rules for 009 – Angular Clock using RxJS & Async Pipe. Guard against null values, empty strings, and layout overflows.

Reference Baseline / Scenario / Code:

```
if (!input || input.trim().length === 0) { displayError("Field required"); }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Validation Rules & Error State: _____

[2] CSS / [ngStyle] Directives: _____

[3] Responsive Breakpoints: _____

EXERCISE 9.4: VERIFICATION & TESTING EXECUTION (DATE PIPE FORMATTING)

Problem Statement: Construct a verification test or manual testing checklist for 009 – Angular Clock using RxJS & Async Pipe. Assert that UI elements render and update correctly upon interaction.

Reference Baseline / Scenario / Code:

```
it('should render element and reflect state change', () => { expect(component).toBeTruthy(); });
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Jasmine / Karma Unit Test: _____

[2] DOM Query Assertions: _____

[3] Manual Verification Steps: _____

SELF-REVIEW & VERIFICATION CHECKLIST:

- [ ] **Template Semantics:** HTML markup uses appropriate semantic elements and ARIA attributes.
- [ ] **Reactive State:** Component state updates smoothly without memory leaks or race conditions.
- [ ] **Validation Coverage:** User inputs properly sanitized, validated, and visually flagged on error.
- [ ] **Directive Optimization:** Structural directives employ trackBy functions where applicable.
- [ ] **Cross-Browser Quality:** Renders cleanly across mobile and desktop viewport sizes.

CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *Why is client-side validation in 009 – Angular Clock using RxJS & Async Pipe essential for UX, even when server-side validation exists?*
- *How does Angular's reactive architecture improve maintainability over manual vanilla JS DOM manipulation?*

Reflection Notes:

_____

_____

COMPLETION STATUS:	EVALUATOR & MENTOR REMARKS:
[ ] COMPLETED (Meets All Solution Criteria) [ ] NEEDS REVISION (Resubmit Corrected Work) **Evaluator Signature:** _____	Feedback Notes: _____ Rubric Evaluation: [ ] Correctness [ ] Code Quality [ ] Edge Cases